

# W1 PRACTICE

## PART 1 – EXPLORATION

 What do you need to **be able to do** for next week?

- Identify the **type** of variable
- Manipulate **nullable** and **non-nullable** variables.
- Differentiate between **final** and **const**.
- Manipulate **strings, lists, and maps**.
- Use **loops** and **conditions** to control flow.
- Define and call functions with positional and **named arguments**, understand **arrow syntax**



**LO NO AI**

Solve problems entirely on your own.

 How to run Dart code?

You can write your code on VS Code, or using this online editor

- [Install Dart and Flutter SDK](#)
- [Online Dart Compiler](#)

 Resources for this research

To help you complete this handout, use the following resources:

- |                               |                                  |                              |
|-------------------------------|----------------------------------|------------------------------|
| ✓ <a href="#">Variables</a>   | ✓ <a href="#">Built-in types</a> | ✓ <a href="#">Conditions</a> |
| ✓ <a href="#">Null Safety</a> | ✓ <a href="#">Lists</a>          | ✓ <a href="#">Functions</a>  |
|                               | ✓ <a href="#">Loops</a>          |                              |



# EX 1 - Type Inference

## [Built-in types](#)

**EXPLAIN** : Explain how Dart infers the type of a variable.

The type of variable in dart is a typical datatype, once we assign var or final we can not change it later on.

**CODE** : Complete the bellow code to illustrate the concepts:

```
// Declare a int variable using var or final and let Dart infer its type
var thisIsNumber = 42;
final stillWithNumber = 32;
// Define a variable with an explicit String type
String thisIsString = "Dina the best"
```

# EX 2 - Nullable and Non-Nullable Variables

## [Variables](#) [Null Safety](#)

**EXPLAIN** : Explain nullable vs non-nullable variables.

By default all the variable are non-nullable meaning we can not give the null value to them. Once we use ( int? integer) "?" meaning we set that integer to be nullable.

**EXPLAIN** : When is it useful to have nullable variables?

Nullable useful when the data can be null and have value. Example the string of middle name, some have their middle name but some are not.

**CODE** : Complete the bellow code to illustrate the concepts:

```
// Declare a nullable integer variable and assign it a null value
Int? nullableInteger = null;
// Declare a non-nullable integer variable and assign it a value
Int nonnull = 10;
// Assign a new value to the nullable variable
nullableInteger = 29;
```

# EX 3 - Final and const

**EXPLAIN** : Describe the difference between final and const.

Final is a type of variable that can assign only once, we determine it as run-time variable.  
Const is a variable type that we can not change the value meaning we hardcoded.

**CODE** : Complete the bellow code to illustrate the concepts:

```
// Declare a final variable and assign it the current date and time
Final datetime now = datetime.now();
```

```
// Can you declare this variable as const? Why? -> We can not use const to declare this datetime because the value change every second.
```

```
// Declare a const variable with a integer value
```

```
Const thisIsConst = 10;
```

```
// Can you reassign the value of this final variable? Why?
```

No, we can not reassign this type because it will cause the run-time error to const.

## EX 4 - Strings, Lists and Maps

[Lists](#) [Built-in types](#)

**CODE :** Complete the bellow code to illustrate the concepts:

### Strings:

```
// Declare two strings: firstName and lastName and an integer:age
```

```
String firstName = "dina";
```

```
String? middleName = null;
```

```
String lastName = "seng"
```

```
Int age = 19;
```

```
// Concatenate the 2 strings and the age
```

```
String information = "$firstName $lastName, Age:, $age";
```

```
// Print result
```

```
Print(information);
```

### Lists:

```
// Create a list of integers
```

```
List<int> numbers = [10,20,30];
```

```
// Add a number to the list
```

```
numbers.add(40);
```

```
// Remove a number from the list
```

```
Numbers.remove(20);
```

```
// Insert a number at a specific index in the list
```

```
Numbers.insert(0,5);
```

```
// Iterate over the list and print each number
```

```
for(var num in numbers) {
```

```
    Print(num);
```

```
}
```

### Maps:

```
// Create a map with String keys and integer values
```

```
Map<string, int> scores = {"Alice": 95, "Masha": 80};
```

```
// Add a new key-value pair to the map
```

```
scores["chacha"] = 88;
```

```
// Remove a key-value pair from the map
```

```
Scores.remove["Alice"];
```

```
// Iterate over the map and print each key-value pair
Scores.forEach((key, value){
    Print("$key scored $value");
});
```

**EXPLAIN :** When should I use a Map instead of a List?

The Map matter when I need to find the value by unique key like every user profile has their own id, so we can just map through the id to get user profile. List we better use it when it has their own order collection – it similar to object.

**EXPLAIN :** When should I use a Set instead of a List?

Set can perform really fast when we don't care about the order meaning that we can find a collection of unique value.

## EX 5 - Loops and Conditions

[Loops](#) [Conditions](#)

**CODE :** Complete the bellow code to illustrate the concepts:

```
// Use a for-loop to print numbers from 1 to 5
For(int i = 0, I < 5, i++){
    Print(i);
}

// Use a while-loop to print numbers while a condition is true
int count = 1;
while(count <= 5) {
    print(count);
    count++;
}

// Use an if-else statement to check if a number is even or odd
Int number = 7 ;
If(number %2 == 0) {
    print("wowww, so smart number is even");
}else{
    Print("hmmm, let ask teacher we don't know even or odd");
}
```

# EX 6 - Functions

## [Functions](#)

**EXPLAIN :** Compare positional and named function arguments

Positional argument reply on the order that we passed. While named argument are passed by their own parameter that name in enclose {}.

**EXPLAIN :** Explain when and how to use arrow syntax for functions?

I think we should use arrow syntax for function when the function it has only 1 expression that need to return.

**CODE :** Complete the bellow code to illustrate the concepts:

### Defining and Invoking a Function:

```
// Define a function that takes two integers and returns their sum
Int num(int a, int b){return a + b;}
// Call the function and print the result
Print("Sum: ${sum(1,1)}");
```

### Positional vs Named Arguments:

```
// Define a function that uses positional arguments
Double calVolume(double length, double width, double height){
Return length*width*height;
}
Print("Volume: ${calVolume(10.0, 5.0,2.0)}");

// Define another function that uses named arguments with the required
keyword (ex: getArea with rectangle arguments)
Double getArea({required double length, required double width}){
    Return length * width;
}
Print("Area: ${getArea(width: 9.0, length: 3.0)}")
// Call both functions with appropriate arguments
```

**EXPLAIN :** Can positional argument be omitted? Show an example

Positional argument can be mark as optional by using [],

Example:

```
Void greet([String name = "dsfs"]){ print(hello, $name)}  
greet();  
greet("dina");
```

**EXPLAIN :** Can named argument be omitted? Show an example

Yes we can but only when we don't mark it as required.

Example:

```
void configureConnection({int timeout = 30, String? protocol}) {  
    print("Timeout: $timeout, Protocol: $protocol");  
}  
configureConnection();
```

**CODE :** Complete the bellow code to illustrate the concepts:

**Arrow Syntax:**

```
// Define a function using arrow syntax that squares a number  
Int square(int n) => n*n;  
// Call the arrow function and print the result  
Print("Square: ${square(4)}");
```

**GitHub Repository:** <https://github.com/dina-seng/mobile-w1.git>